Nume: Economu Victor, Radu Alexandru-Mihail

Grupa: 506

Specificatia problemei

Analiza evolutiei scorurilor unui participant

1. Descrierea problemei

In cadrul unei platforme educationale, un participant obtine mai multe scoruri la o serie de teste, sustinute in ordine cronologica.

Se considera un vector de numere intregi care contine scorurile obtinute de participant, in ordinea sustinerii testelor.

Scopul programului este identificarea unor perioade de crestere consecutiva a scorurilor si verificarea indeplinirii unor conditii prestabilite.

2. Date de intrare

Programul primeste un vector de numere intregi `scores`, care trebuie sa respecte urmatoarele conditii:

- Referinta catre vector nu poate fi `null`.
- Vectorul trebuie sa contina intre 6 si 12 elemente, inclusiv.
- Fiecare element reprezinta un scor intreg cuprins intre 0 si 100, inclusiv.
- Elementele sunt ordonate cronologic.

3. Definitia unei secvente relevante

O secventa maximală strict crescatoare reprezinta o succesiune de elemente consecutive din vector, in care fiecare element este strict mai mare decat precedentul.

Secventa este maximala daca nu poate fi extinsa la stanga sau la dreapta, pastrand ordinea strict crescatoare.

Se considera relevante numai secventele maximale care contin cel putin 3 elemente.

4. Conditiile de verificat

Programul determina daca sunt indeplinite simultan urmatoarele conditii:

1. Exista cel putin doua secvente maximale strict crescatoare relevante.
2. Suma ultimelor scoruri ale tuturor secventelor relevante este mai mare sau egala cu 80.

5. Date de iesire

Metoda returneaza un rezultat de tip `String`:

- `"DA"` – ambele conditii sunt indeplinite.
- `"NU"` – vectorul este valid, dar cel putin una dintre conditii nu este indeplinita.
- `"Vectorul nu este valid"` – referinta este nula, dimensiunea vectorului este invalida sau exista cel putin un scor in afara intervalului permis.

6. Exemplu

Pentru vectorul:

`[10, 30, 50, 0, 20, 70]`

Se identifica doua secvente maximale strict crescatoare relevante:

- `[10, 30, 50]` – lungime 3, ultimul scor 50.
- `[0, 20, 70]` – lungime 3, ultimul scor 70.

Numarul secventelor relevante este 2.

Suma ultimelor scoruri este 50 + 70 = 120.

Deoarece sunt indeplinite ambele conditii, programul returneaza `"DA"`.

7. Precizari

- Programul nu modifica vectorul primit.
- Sunt analizate toate secventele maximale strict crescatoare.
- Secventele cu lungime mai mica de 3 nu contribuie la numarul secventelor relevante sau la suma calculata.
- Pentru un vector invalid, programul returneaza mesajul de eroare fara a efectua analiza secventelor.

Cerinta 1.a - Equivalence Partitioning

Partitionarea in clase de echivalenta

Se considera metoda `IncreasingSequenceAnalyzer.analyze(int[] scores)`, care analizeaza evolutia scorurilor unui participant.

Metoda primeste un vector de scoruri si determina daca exista cel putin doua secvente maximale strict crescatoare, fiecare cu lungime de minimum 3, iar suma ultimelor elemente ale tuturor acestor secvente este cel putin 80.

Un vector este valid daca are intre 6 si 12 elemente, iar fiecare element apartine intervalului $[0,100]$.

Clasele datelor de intrare

Notam cu v vectorul de intrare, cu $¿v \vee ¿$ lungimea acestuia si cu $v[i]$ elementul de pe pozitia i.

Definim urmatoarele clase:

$N_1 = \{v \vee v \neq null, 6 \leq \vee v \vee \leq 12, \forall\ i : 0 \leq v[i] \leq 100\}$

$N_2 = \{v \vee v = null\}$

$N_3 = \{v \vee v \neq null, \vee v \vee 6\}$

$N_4 = \{v \vee v \neq null, \vee v \vee 12\}$

$N_5 = \{v \vee v \neq null, 6 \leq \vee v \vee \leq 12, \exists\ i : v[i] < 0\}$

$N_6 = \{v \vee v \neq null, 6 \leq \vee v \vee \leq 12, \forall\ i : v[i] \geq 0, \exists\ i : v[i] > 100\}$

Indicii i sunt considerati in intervalul $0 \leq i < v \vee$.

- $\mathbf{N_1}$ – vector valid.
- $\mathbf{N_2}$ – referinta nula.
- $\mathbf{N_3}$ – lungime mai mica de 6.
- $\mathbf{N_4}$ – lungime mai mare de 12.
- $\mathbf{N_5}$ – lungime valida, dar exista cel putin un scor negativ.
- $\mathbf{N_6}$ – lungime valida, fara scoruri negative, dar exista cel putin un scor mai mare decat 100.

Pentru vectorii din N_1, definim urmatoarele proprietati:

$P_1(v) : k(v) \geq 2$

$P_2(v) : S(v) \geq 80$

unde:

- $k(v)$ reprezinta numarul secventelor maximale strict crescatoare, cu lungime de minimum 3.
- $S(v)$ reprezinta suma ultimelor elemente ale tuturor acestor secvente. Daca nu exista secvente relevante, $S(v) = 0$.

Clasele datelor de iesire

Definim clasele datelor de iesire:

$I_1 = \{\ DA\ \}$

$I_2 = \{ NU \}$

$I_3 = \{ Vectorul\ nu\ este\ valid \}$

Clasele de echivalenta finale

Combinand clasele datelor de intrare cu proprietatile definite si cu rezultatele posibile ale metodei, obtinem:

$C_1 = \{ v \in N_1 \vee P_1(v) \wedge P_2(v) \} \rightarrow I_1$

$C_2 = \{ v \in N_1 \vee P_1(v) \wedge \neg P_2(v) \} \rightarrow I_2$

$C_3 = \{ v \in N_1 \vee \neg P_1(v) \wedge P_2(v) \} \rightarrow I_2$

$C_4 = \{ v \in N_1 \vee \neg P_1(v) \wedge \neg P_2(v) \} \rightarrow I_2$

$C_5 = N_2 \rightarrow I_3$

$C_6 = N_3 \rightarrow I_3$

$C_7 = N_4 \rightarrow I_3$

$C_8 = N_5 \rightarrow I_3$

$C_9 = N_6 \rightarrow I_3$

Au fost obtinute 9 clase de echivalenta, disjuncte si care acopera intregul domeniu de intrare.

Datele de test

Pentru fiecare clasa de echivalenta a fost ales un reprezentant.

Test	Clasa	Date de intrare	Rezultat asteptat
T1	C_1	`[10,30,50,0,20,70]`	DA
T2	C_2	`[10,20,30,0,10,40]`	NU
T3	C_3	`[10,20,90,80,70,60]`	NU
T4	C_4	`[0,0,0,0,0,0]`	NU
T5	C_5	`null`	Vectorul nu este valid
T6	C_6	`[10,20,30,0,20]`	Vectorul nu este valid
T7	C_7	`[0,1,2,3,4,5,6,7,8,9,10,11,12]`	Vectorul nu este valid
T8	C_8	`[10,30,50,0,20,-1]`	Vectorul nu este valid
T9	C_9	`[10,30,50,0,20,101]`	Vectorul nu este valid

Pentru testele care utilizeaza vectori valizi, se obtin urmatoarele valori:

Test	$k(v)$	$S(v)$	$P_1(v)$	$P_2(v)$
T1	2	120	True	True

Test	$k(v)$	$S(v)$	$P_1(v)$	$P_2(v)$
T2	2	70	True	False
T3	1	90	False	True
T4	0	0	False	False

Cerinta 1.b - Boundary Value Analysis

Analiza valorilor de frontiera

Se identifica urmatoarele conditii si valori de frontiera:

1. Lungimea vectorului

$6 \leq n \leq 12$

$n \in \{5, 6, 7, 11, 12, 13\}$

2. Valorile elementelor vectorului

$0 \leq v[i] \leq 100$

$v[i] \in \{-1, 0, 1, 99, 100, 101\}$

3. Lungimea unei secvente relevante

$l \geq 3$

$l \in \{2, 3, 4\}$

4. Numarul secventelor relevante

$k \geq 2$

$k \in \{1, 2, 3\}$

5. Suma ultimelor elemente ale secventelor relevante

$S \geq 80$

$S \in \{79, 80, 81\}$

6. Conditia de crestere stricta

$v[i] > v[i-1]$

Notam diferenta dintre doua elemente consecutive cu:

$d = v[i] - v[i-1]$

$d \in \{-1, 0, 1\}$

Datele de test

A. Lungimea vectorului

Test	Valoare	Date de intrare	Rezultat asteptat
T1	n = 5	`[0,0,0,0,0]`	Vectorul nu este valid
T2	n = 6	`[0,0,0,0,0,0]`	NU
T3	n = 7	`[0,0,0,0,0,0,0]`	NU
T4	n = 11	`[0,0,0,0,0,0,0,0,0,0,0]`	NU
T5	n = 12	`[0,0,0,0,0,0,0,0,0,0,0,0]`	NU
T6	n = 13	`[0,0,0,0,0,0,0,0,0,0,0,0,0]`	Vectorul nu este valid

B. Limita inferioara a scorurilor

Test	Valoare	Date de intrare	Rezultat asteptat
T7	-1	`[-1,20,40,0,20,40]`	Vectorul nu este valid
T8	0	`[0,20,40,0,20,40]`	DA
T9	1	`[1,20,40,0,20,40]`	DA

C. Limita superioara a scorurilor

Test	Valoare	Date de intrare	Rezultat asteptat
T10	99	`[0,20,40,0,20,99]`	DA
T11	100	`[0,20,40,0,20,100]`	DA
T12	101	`[0,20,40,0,20,101]`	Vectorul nu este valid

D. Lungimea secventelor relevante

Test	Valoare	Date de intrare	Rezultat asteptat
T13	l = 2	`[0,20,40,0,40,0]`	NU
T8	l = 3	`[0,20,40,0,20,40]`	DA
T14	l = 4	`[0,10,20,40,0,20,40]`	DA

Pentru T13 exista o secventa relevanta de lungime 3 si o secventa de lungime 2, care nu este considerata relevanta. Rezultatul este `"NU"`.

Pentru T8 exista doua secvente relevante de lungime 3, iar pentru T14 exista doua secvente relevante de lungimi 4 si 3.

E. Numarul secventelor relevante

Test	Valoare	Date de intrare	Rezultat asteptat
T15	k = 1	`[0,20,80,0,0,0]`	NU
T8	k = 2	`[0,20,40,0,20,40]`	DA
T16	k = 3	`[0,10,30,0,10,30,0,10,30]`	DA

Pentru T15 exista o singura secventa relevanta, iar suma este 80.

Pentru T8 exista doua secvente relevante, cu suma 80.

Pentru T16 exista trei secvente relevante, cu suma 90.

F. Suma ultimelor elemente ale secventelor relevante

Test	Valoare	Date de intrare	Rezultat asteptat
T17	S = 79	`[0,10,39,0,10,40]`	NU
T8	S = 80	`[0,20,40,0,20,40]`	DA
T18	S = 81	`[0,10,41,0,10,40]`	DA

Toate cele trei teste contin doua secvente relevante. Rezultatul depinde de compararea sumei ultimelor elemente cu pragul 80.

G. Conditia de crestere stricta

Test	Valoare	Date de intrare	Rezultat asteptat
T19	d = -1	`[0,40,80,0,20,19]`	NU
T20	d = 0	`[0,40,80,0,20,20]`	NU
T21	d = 1	`[0,40,80,0,20,21]`	DA

Pentru T19, ultimul scor este mai mic decat precedentul, iar pentru T20 cele doua scoruri sunt egale. In ambele cazuri exista o singura secventa relevanta.

Pentru T21, ultima pereche este strict crescatoare. Exista doua secvente relevante, iar suma ultimelor elemente este 101.

Cerinta 1.c - Cause-Effect Graphing

Graful cauza-efect

1. Identificarea cauzelor

Se definesc urmatoarele cauze:

- **C_1:** vectorul este nenul.
- **C_2:** lungimea vectorului este cel putin 6.
- **C_3:** lungimea vectorului este cel mult 12.
- **C_4:** toate elementele vectorului sunt mai mari sau egale cu 0.
- **C_5:** toate elementele vectorului sunt mai mici sau egale cu 100.
- **C_6:** exista cel putin doua secvente maximale strict crescatoare, fiecare cu lungime de minimum 3.
- **C_7:** suma ultimelor elemente ale tuturor secventelor relevante este cel putin 80.

Notand vectorul cu v, numarul secventelor relevante cu $k(v)$ si suma ultimelor elemente cu $S(v)$, cauzele pot fi exprimate astfel:

$C_1: v \neq null$

$C_2: \vee\ v \vee \geq 6$

$C_3: \vee\ v \vee \leq 12$

$C_4: \forall\ i, v[i] \geq 0$

$C_5: \forall\ i, v[i] \leq 100$

$C_6: k(v) \geq 2$

$C_7: S(v) \geq 80$

Cauzele C_2–C_5 se aplica vectorilor nenuli, iar C_6 si C_7 se evalueaza numai pentru vectorii valizi.

2. Identificarea efectelor

Se definesc urmatoarele efecte:

- **E_1:** metoda returneaza `"DA"`.
- **E_2:** metoda returneaza `"NU"`.
- **E_3:** metoda returneaza `"Vectorul nu este valid"`.

3. Relatiile dintre cauze si efecte

Conditia de validitate a vectorului este:

$V = C_1 \wedge C_2 \wedge C_3 \wedge C_4 \wedge C_5$

Conditia de acceptare pentru un vector valid este:

$B = C_6 \wedge C_7$

Efectele se exprima astfel:

$E_1 = V \wedge B$

$E_2 = V \wedge \neg B$

$E_3 = \neg V$

Prin inlocuirea expresiilor intermediare se obtin:

$E_1 = C_1 \wedge C_2 \wedge C_3 \wedge C_4 \wedge C_5 \wedge C_6 \wedge C_7$

$E_2 = C_1 \wedge C_2 \wedge C_3 \wedge C_4 \wedge C_5 \wedge (\neg C_6 \vee \neg C_7)$

$E_3 = \neg C_1 \vee \neg C_2 \vee \neg C_3 \vee \neg C_4 \vee \neg C_5$

Negarea uneia dintre cauzele C_1–C_5 determina invaliditatea vectorului.

Pentru un vector valid, efectul E_1 este produs numai daca ambele cauze C_6 si C_7 sunt adevarate. In caz contrar, se produce efectul E_2.

4. Constrangeri intre cauze

Cauzele C_2–C_5 necesita existenta vectorului. Daca C_1 este falsa, acestea nu sunt aplicabile.

C_2 si C_3 nu pot fi simultan false pentru un vector nenul:

$$C_1 \Rightarrow (C_2 \vee C_3)$$

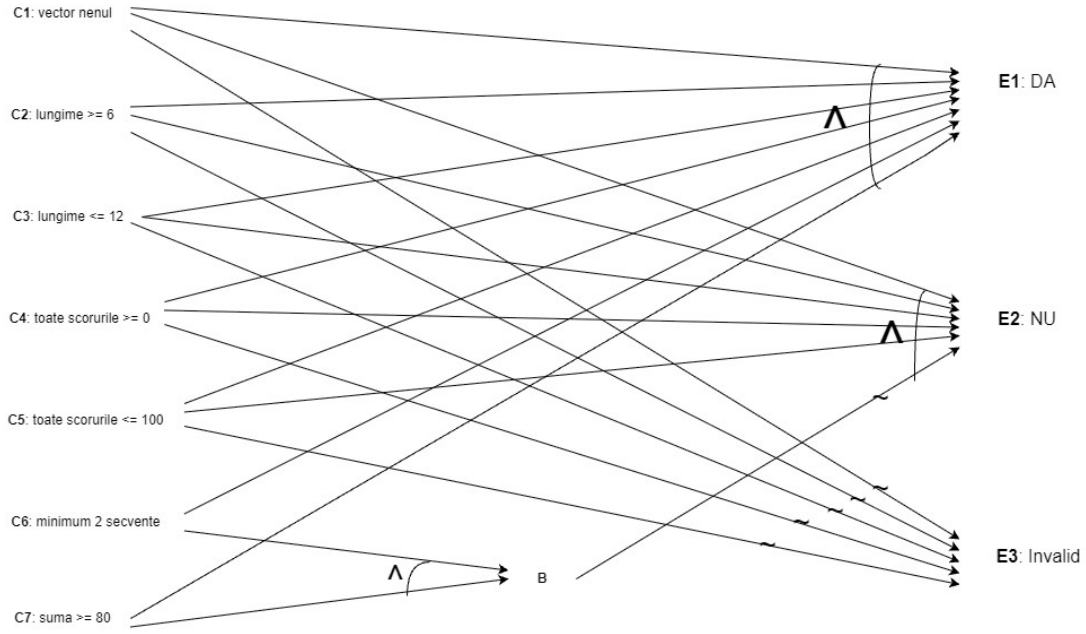
Cauzele C_4 si C_5 pot fi simultan false, deoarece un vector poate contine atat un scor negativ, cat si un scor mai mare decat 100.

Cauzele C_6 si C_7 sunt aplicabile numai daca vectorul este valid:

$$V = 1$$

Pentru vectorii valizi, toate cele patru combinatii ale cauzelor C_6 si C_7 sunt realizabile.

5. Reprezentarea grafului cauza-efect



Graful cauza-efect

6. Tabelul de decizie

Pe baza grafului cauza-efect si a constrangerilor sunt obtinute urmatoarele reguli.

Notatii:

- **1** – cauza sau efect adevarat.

- **0** – cauza sau efect fals.
- **–** – cauza neaplicabila sau a carei valoare nu influenteaza efectul.

Cauze / Efecte	R2	R3	R4	R5	R6	R7	R8	R9
C_1	1	1	1	1	1	1	1	1
C_2	0	1	1	1	1	1	1	1
C_3	1	0	1	1	1	1	1	1
C_4	–	–	0	1	1	1	1	1
C_5	–	–	–	0	1	1	1	1
C_6	–	–	–	–	1	1	0	0
C_7	–	–	–	–	1	0	1	0
E_1 – DA	0	0	0	0	1	0	0	0
E_2 – NU	0	0	0	0	0	1	1	1
E_3 – Invalid	1	1	1	1	0	0	0	0

Regulile corespund urmatoarelor situatii:

- **R1:** vectorul este `null` ($C_1 = 0$).
- **R2:** vectorul este nenul, dar lungimea sa este mai mica de 6 ($C_2 = 0$).
- **R3:** vectorul este nenul, dar lungimea sa este mai mare de 12 ($C_3 = 0$).
- **R4:** vectorul are lungime valida, dar contine cel putin un scor negativ ($C_4 = 0$).
- **R5:** vectorul are lungime valida, nu contine scoruri negative, dar contine cel putin un scor peste 100 ($C_5 = 0$).
- **R6:** vectorul este valid, iar $C_6 = 1$ si $C_7 = 1$.
- **R7:** vectorul este valid, iar $C_6 = 1$ si $C_7 = 0$.
- **R8:** vectorul este valid, iar $C_6 = 0$ si $C_7 = 1$.
- **R9:** vectorul este valid, iar $C_6 = 0$ si $C_7 = 0$.

Regulile R1–R5 acopera toate situatiile de invaliditate, iar R6–R9 acopera toate combinatiile realizabile ale cauzelor C_6 si C_7 pentru vectorii valizi.

7. Generarea datelor de test

Pentru fiecare regula din tabelul de decizie se selecteaza un caz de test reprezentativ.

Test	Regula	Date de intrare	Rezultat asteptat
T1	R1	`null`	Vectorul nu este valid
T2	R2	`[1,2,3,4]`	Vectorul nu este valid
T3	R3	`[0,0,0,0,0,0,0,0,0,0,0,0,0]`	Vectorul nu este valid
T4	R4	`[5,10,20,0,-1,40]`	Vectorul nu este valid
T5	R5	`[0,20,40,0,20,101]`	Vectorul nu este valid

Test	Regula	Date de intrare	Rezultat asteptat
T6	R6	`[0,10,30,0,10,30,0,10,30]`	DA
T7	R7	`[0,10,35,0,10,44]`	NU
T8	R8	`[10,20,100,90,80,70]`	NU
T9	R9	`[40,40,40,40,40,40]`	NU

8. Verificarea datelor de test

Pentru vectorii valizi, notam cu $k(v)$ numarul secventelor relevante si cu $S(v)$ suma ultimelor elemente ale acestora.

Test	$k(v)$	$S(v)$	C_6	C_7	Efect
T6	3	90	1	1	E_1
T7	2	79	1	0	E_2
T8	1	100	0	1	E_2
T9	0	0	0	0	E_2

T1: vectorul este `null`, deci C_1 este falsa si se produce efectul E_3.

T2: vectorul contine 4 elemente, deci C_2 este falsa si se produce efectul E_3.

T3: vectorul contine 13 elemente, deci C_3 este falsa si se produce efectul E_3.

T4: vectorul contine un scor negativ (-1), deci C_4 este falsa si se produce efectul E_3.

T5: vectorul contine un scor peste limita maxima (101), deci C_5 este falsa si se produce efectul E_3.

T6: exista trei secvente relevante de forma `[0,10,30]`. Suma ultimelor elemente este 90. Toate cele sapte cauze sunt adevarate, deci se produce efectul E_1.

T7: exista doua secvente relevante, `[0,10,35]` si `[0,10,44]`. Suma ultimelor elemente este 79. Cauza C_6 este adevarata, iar C_7 este falsa, deci se produce efectul E_2.

T8: exista o singura secventa relevanta, `[10,20,100]`, iar suma este 100. Cauza C_6 este falsa, iar C_7 este adevarata, deci se produce efectul E_2.

T9: vectorul contine doar valori egale. Nu exista secvente relevante, iar suma este 0. Cauzele C_6 si C_7 sunt false, deci se produce efectul E_2.

Cerinta 2 - Code Coverage

Analiza acoperirii codului

Pentru masurarea nivelului de acoperire a codului obtinut prin cele trei metode de testare functionala de la cerinta 1, a fost utilizat instrumentul **JaCoCo 0.8.16**, integrat in proiectul Maven.

Au fost analizate urmatoarele seturi de teste:

- **Equivalence Partitioning (EP)** - 9 teste.
- **Boundary Value Analysis (BVA)** - 21 de teste.
- **Cause-Effect Graphing (CEG)** - 9 teste.

Masuratorile au fost realizate separat pentru fiecare set, asupra clasei `IncreasingSequenceAnalyzer`, care contine metoda `analyze()` si functia auxiliara `getIncreasingSequenceLength()`.

1. Executarea testelor

JaCoCo a generat rapoarte HTML si CSV, din care au fost extrase valorile pentru acoperirea instructiunilor, ramurilor, liniilor si metodelor. Toate testele executate au trecut cu succes, fara erori sau esecuri.

Pentru fiecare set au fost utilizate urmatoarele comenzi:

- EP: `mvn clean verify "-Dtest=EquivalencePartitioningTest"`
- BVA: `mvn clean verify "-Dtest=BoundaryValueAnalysisTest"`
- CEG: `mvn clean verify "-Dtest=CauseEffectGraphingTest"`

Comanda `clean` elimina rezultatele executiei precedente, iar parametrul `-Dtest` selecteaza doar setul analizat. Deci, masuratorile JaCoCo nu sunt cumulate.

2. Rezultatele obtinute

Rezultatele masuratorilor sunt prezentate in tabelul urmator:

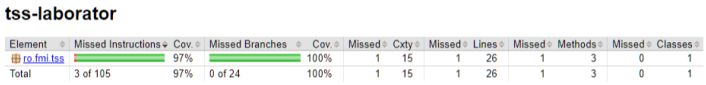
Indicator	EP	BVA	CEG
Numar teste	9	21	9
Instruction Coverage	102/105 (97%)	100/105 (95%)	102/105 (97%)
Branch Coverage	24/24 (100%)	23/24 (95%)	24/24 (100%)
Line Coverage	25/26	24/26	25/26
Method Coverage	2/3	2/3	2/3
Teste esuate	0	0	0

Procentele din tabel sunt cele afisate in **rapoartele JaCoCo**.

Instruction Coverage = procentul instructiunilor bytecode executate de teste.
Branch Coverage = procentul ramurilor parcurse in timpul executiei.
Line Coverage = procentul liniilor executabile acoperite.
Method Coverage = procentul metodelor executate, inclusiv constructorii.

3. Rapoartele JaCoCo

Equivalence Partitioning:

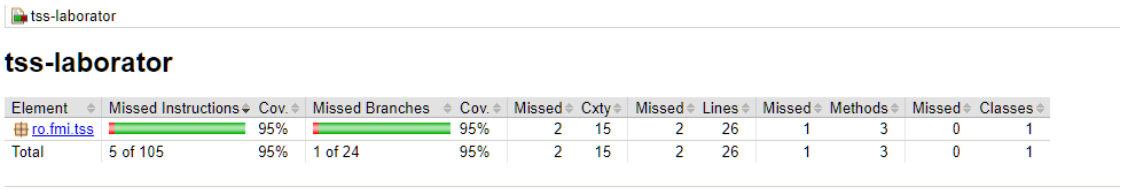


tss-laborator

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
ro.fmi.tss		97%		100%	1	15	1	26	1	3	0	1
Total	3 of 105	97%	0 of 24	100%	1	15	1	26	1	3	0	1

Raport JaCoCo EP

Boundary Value Analysis:



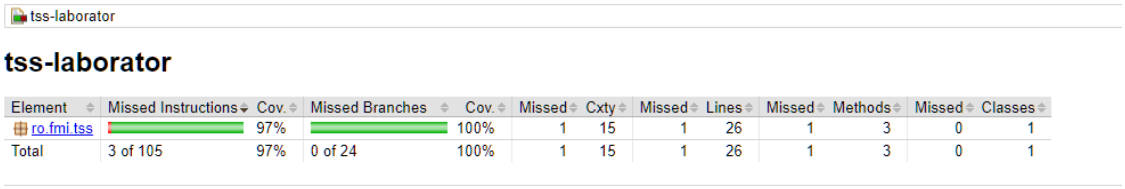
tss-laborator

tss-laborator

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
ro.fmi.tss		95%		95%	2	15	2	26	1	3	0	1
Total	5 of 105	95%	1 of 24	95%	2	15	2	26	1	3	0	1

Raport JaCoCo BVA

Cause-Effect Graphing:



tss-laborator

tss-laborator

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
ro.fmi.tss		97%		100%	1	15	1	26	1	3	0	1
Total	3 of 105	97%	0 of 24	100%	1	15	1	26	1	3	0	1

Raport_JaCoCo_CEG

Datele corespunzatoare fiecarui raport au fost salvate separat in fisierele `docs/coverage/EP.csv`, `docs/coverage/BVA.csv` si `docs/coverage/CEG.csv`.

4. Compararea rezultatelor

Seturile EP si CEG au obtinut rezultate identice, cu 97% Instruction Coverage si 100% Branch Coverage.

Desi au fost construite prin tehnici diferite, ambele seturi verifica principalele categorii de invaliditate si combinatiile conditiilor finale ale programului.

Setul BVA, desi contine mai multe teste, a obtinut 95% Instruction Coverage si 95% Branch Coverage.

Diferenta provine din absenta unui caz de test cu referinta `null` in setul BVA.

La inceputul metodei `analyze()` exista conditia:

```
if (scores == null) {
    return "Vectorul nu este valid";
}
```

Toate cele 21 de teste BVA transmit vectori nenuli. Prin urmare, ramura corespunzatoare conditiei `scores == null` adevarate nu este executata.

In schimb, seturile EP si CEG contin cate un test cu intrare `null`, acoperind si aceasta ramura.

Raportul detaliat JaCoCo pentru BVA evidentiaza conditia `scores == null` prin marcajul galben (acoperire partiala), iar instructiunea `return` corespunzatoare este marcata cu rosu (neexecutata).

IncreasingSequenceAnalyzer.java

```java
package ro.fmi.tss;

public class IncreasingSequenceAnalyzer {

    private static int getIncreasingSequenceLength(int[] scores, int start) {
        int len = 1;

        for (int i = start + 1; i < scores.length; i++) {
            if (scores[i] > scores[i - 1]) {
                len++;
            } else {
                break;
            }
        }

        return len;
    }

    public static String analyze(int[] scores) {
        if (scores == null) {
            return "Vectorul nu este valid";
        }

        if (scores.length < 6 || scores.length > 12) {
            return "Vectorul nu este valid";
        }

        for (int i = 0; i < scores.length; i++) {
            if (scores[i] < 0 || scores[i] > 100) {
                return "Vectorul nu este valid";
            }
        }

        int seqCount = 0;
        int sum = 0;

        for (int start = 0; start < scores.length;) {
            int len = getIncreasingSequenceLength(scores, start);

            if (len >= 3) {
                seqCount++;
                int last = start + len - 1;
                sum += scores[last];
            }

            start += len;
        }

        if (seqCount >= 2 && sum >= 80) {
            return "DA";
        }

        return "NU";
    }
}
```

Ramura neacoperita de BVA

Astfel, BRANCH COVERAGE-ul este:

$$BC_{EP} = \frac{24}{24} \cdot 100 = 100\,\%$$

$$BC_{BVA} = \frac{23}{24} \cdot 100 \approx 95.83\,\%$$

$$BC_{CEG} = \frac{24}{24} \cdot 100 = 100\,\%$$

JaCoCo afiseaza valoarea de 95% pentru BVA.

Diferenta de o ramura explica si cele doua instructiuni suplimentare si linia suplimentara neacoperite de setul BVA.

5. Analiza metodelor neacoperite

Toate cele trei seturi au obtinut Method Coverage de 2/3.

Cele doua metode executate sunt:

- `analyze()`
- `getIncreasingSequenceLength()`

A treia metoda raportata de JaCoCo este constructorul implicit al clasei `IncreasingSequenceAnalyzer`, generat automat de compilatorul Java deoarece clasa nu declara explicit un constructor.

Bytecode-ul constructorului contine trei instructiuni: `aload_0`, `invokespecial` si `return`.

```
public ro.fmi.tss.IncreasingSequenceAnalyzer();
  Code:
     0: aload_0
     1: invokespecial #1                  // Method java/lang/Object."<init>":()V
     4: return
```

Bytecode constructor implicit

Testele utilizeaza exclusiv metodele statice, fara sa creeze instante ale clasei `IncreasingSequenceAnalyzer`.

Prin urmare, constructorul implicit nu este executat. Acesta explica cele 3 instructiuni si metoda neacoperite in rapoartele EP si CEG.

6. Concluzie

EP si CEG au obtinut cea mai buna acoperire structurala, cu 100% Branch Coverage si 97% Instruction Coverage.

BVA a obtinut 95% Branch Coverage si 95% Instruction Coverage, deoarece nu include un test pentru referinta nula.

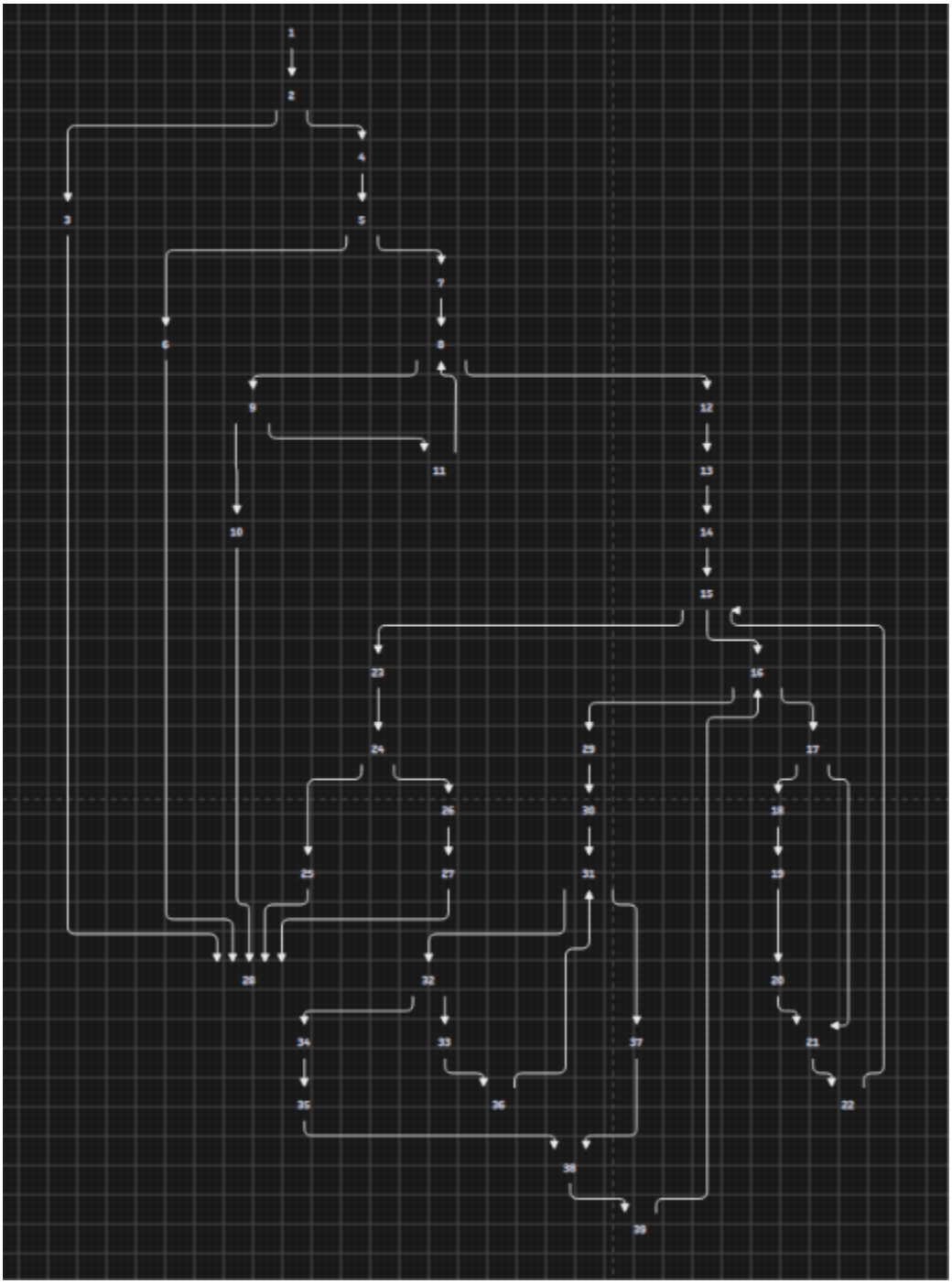
Rezultatele arata ca un numar mai mare de teste nu garanteaza o acoperire structurala mai ridicata, aceasta depinzand de ramurile si instructiunile efectiv executate. Acoperirea obtinuta nu demonstreaza absenta tuturor defectelor, ci indica masura in care seturile de teste exercita structura programului.

Implementarea programului în cod

```java
public static String analyze(int[] scores) {
    if (scores == null) {
        return "Vectorul nu este valid";
    }
    if (scores.length < 6 || scores.length > 12) {
        return "Vectorul nu este valid";
    }
    for (int i = 0; i < scores.length; i++) {
        if (scores[i] < 0 || scores[i] > 100) {
            return "Vectorul nu este valid";
        }
    }
    int seqCount = 0;
    int sum = 0;
    for (int start = 0; start < scores.length;) {
        int len = getIncreasingSequenceLength(scores, start);
        if (len >= 3) {
            seqCount++;
            int last = start + len - 1;
            sum += scores[last];
        }
        start += len;
    }
    if (seqCount >= 2 && sum >= 80) {
        return "DA";
    }
    return "NU";
}
private static int getIncreasingSequenceLength(int[] scores, int start) {
    int len = 1;
    for (int i = start + 1; i < scores.length; i++) {
        if (scores[i] > scores[i - 1]) {
            len++;
        } else {
            break;
        }
    }
    return len;
}
```

Exercițiul 3

Graful orientat al problemei



Calculul Complexitatii Ciclomatice

Numarul total de noduri (V): 39
Numarul total de arce (E): 47
Numarul de componente conexe (P): 1

V(G) = E - V + 2xP = 47 - 39 + 2x1 = 8 + 2 = 10
Setul de Circuite Liniar Independente

- **C1 (Se oprește la referință nulă)**: 1 → 2 → 3 → 28
- **C2 (Se oprește la lungime invalidă a vectorului)**: 1 → 2 → 4 → 5 → 6 → 28
- **C3 (Evită ambele bucle - drum matematic de bază)**: 1 → 2 → 4 → 5 → 7 → 8 → 12 → 13 → 14 → 15 → 23 → 24 → 26 → 27 → 28
- **C4 (Găsește scor invalid în prima buclă și iese)**: 1 → 2 → 4 → 5 → 7 → 8 → 9 → 10 → 28
- **C5 (Scor valid în prima buclă, continuă iterarea)**: 1 → 2 → 4 → 5 → 7 → 8 → 9 → 11 → 8 → 12 → *(continuă ca C3)*
- **C6 (Apelează metoda ajutătoare, dar nu intră în bucla ei internă)**: 1 → ... → 15 → 16 → 29 → 30 → 31 → 37 → 38 → 39 → 17 → 21 → 22 → 15 → 23 → *(continuă ca C3)*
- **C7 (În interiorul metodei ajutătoare, intră pe ramura `else` cu `break`)**: 1 → ... → 31 → 32 → 34 → 35 → 38 → 39 → 17 → 21 → 22 → 15 → 23 → *(continuă ca C3)*
- **C8 (În interiorul metodei ajutătoare, intră pe ramura `if` și crește secvența)**: 1 → ... → 31 → 32 → 33 → 36 → 31 → 37 → 38 → 39 → 17 → 21 → 22 → 15 → 23 → *(continuă ca C3)*
- **C9 (Satisface condiția `len >= 3` în bucla principală și procesează secvența)**: 1 → ... → 16 → ... → 39 → 17 → 18 → 19 → 20 → 21 → 22 → 15 → 23 → *(continuă ca C3)*
- **C10 (Satisface condiția finală de validare globală și returnează "DA")**: 1 → ... → 15 → 23 → 24 → 25 → 28

Exercițiul 4

Decizia 1

if (scores.length < 6 || scores.length > 12)

Condiții individuale:

C1: scores.length < 6

C2: scores.length > 12

Decizia **D** = **(scores.length < 6 || scores.length > 12)** = C1 v C2

- Acoperire la nivel de instructiune D = true, D = false
- Acoperire la nivel de decizie D = true, D = false
  - D = true : C1 = true, C2 = false
  - D = false: C1 = false, C2 = false
- Acoperire la nivel de conditie: C1 = true/false si C2 = true/false
  - C1 = true si C2 = false => D = true
  - C1 = false si C2 = true => D = true

3. Acoperire la nivel de decizie/conditie C1 = true/false si C2 = true/false, D = true/false
   1. D = true: C1 = false, C2 = true
   2. D = true: C1 = true, C2 = false
   3. D = false: C1 = false, C2 = false

- Acoperire la nivel de conditii multiple:
  - D = true: C1 = true, C2 = true (varianta imposibila)
  - D = false: C1 = false, C2 = false
  - D = true: C1 = true, C2 = false
  - D = true: C1 = false, C2 = true
- Acoperire la nivel de conditie/decizie modificata: fiecare conditie individuala sa fie atat true cat si false: C1 = true/false, C2 = true/false; fiecare decizie este true/false: D = true/false si fiecare conditie individuala influenteaza independent decizia.
  - D = true: C1 = true, C2 = false => [10, 20, 30, 40, 50] (5 elemente) (T1)
  - D = true: C1 = false, C2 = true => [1, 2, 3, … , 13] (13 elemente) (T2)
  - D = false: C1 = false, C2 = false => [10, 20, 50, 5, 15, 30] (6 elemente) (T3)

	C1	C2	D1

T1	true	false	true
T2	false	true	true
T3	false	false	false

Decizia 2

if (scores[i] < 0 || scores[i] > 100)

Conditii individuale:

C1: scores[i] < 0

C2: scores[i] > 100

Decizia **D** = **(scores[i] < 0 || scores[i] > 100)** = C1 v C2

- Acoperire la nivel de instructiune D = true, D = false
- Acoperire la nivel de decizie D = true, D = false
  - D = true : C1 = true, C2 = false
  - D = false: C1 = false, C2 = false
- Acoperire la nivel de conditie: C1 = true/false si C2 = true/false
  - C1 = true si C2 = false => D = true
  - C1 = false si C2 = true => D = true

4. Acoperire la nivel de decizie/conditie C1 = true/false si C2 = true/false, D = true/false
   1. D = true: C1 = false, C2 = true
   2. D = true: C1 = true, C2 = false
   3. D = false: C1 = false, C2 = false

- Acoperire la nivel de conditii multiple:
  - D = true: C1 = true, C2 = true (varianta imposibila)
  - D = false: C1 = false, C2 = false
  - D = true: C1 = true, C2 = false
  - D = true: C1 = false, C2 = true
- Acoperire la nivel de conditie/decizie modificata: fiecare conditie individuala sa fie atat true cat si false: C1 = true/false, C2 = true/false; fiecare decizie este true/false: D = true/false si fiecare conditie individuala influenteaza independent decizia.
  - D = true: C1 = true, C2 = false => [10, 20, -5, 40, 50, 60] (T4)
  - D = true: C1 = false, C2 = true => [10, 20, 105, 40, 50, 60] (T5)
  - D = false: C1 = false, C2 = false => T3 acopera T6

	C1	C2	D2

T4	true	false	true
T5	false	true	true
T6	false	false	false

Decizia 3

if (seqCount >= 2 && sum >= 80)

Conditii individuale:

C1: seqCount >= 2

C2: sum >= 80

Decizia **D** = **(seqCount >= 2 && sum >= 80)** = C1 ^ C2

- Acoperire la nivel de instructiune D = true, D = false
- Acoperire la nivel de decizie D = true, D = false
  - D = true : C1 = true, C2 = true
  - D = false: C1 = true, C2 = false
- Acoperire la nivel de conditie: C1 = true/false si C2 = true/false
  - C1 = true si C2 = false => D = false
  - C1 = false si C2 = true => D = false

5. Acoperire la nivel de decizie/conditie C1 = true/false si C2 = true/false, D = true/false
   1. D = true: C1 = true, C2 = true
   2. D = false: C1 = false, C2 = false

- Acoperire la nivel de conditii multiple:
  - D = true: C1 = true, C2 = true
  - D = false: C1 = false, C2 = false
  - D = false: C1 = true, C2 = false
  - D = false: C1 = false, C2 = true
- Acoperire la nivel de conditie/decizie modificata: fiecare conditie individuala sa fie atat true cat si false: C1 = true/false, C2 = true/false; fiecare decizie este true/false: D = true/false si fiecare conditie individuala influenteaza independent decizia.
  - D = true: C1 = true, C2 = true => T3 acopera T7
  - D = false: C1 = false, C2 = true => [10, 20, 30, 40, 50, 90](1 sec,sum=90)(T8)
  - D = false: C1 = true, C2 = false => [10, 20, 30, 5, 10, 20](2 sec, sum=50)(T9)

	C1	C2	D3

T7	true	true	true
T8	false	true	false
T9	true	false	false

Decizia 4

if (scores == null)

Condiția individuală:

C: scores == null

	C	D4
T10	true	true
T11	false	false

T10: null
T3 acopera T11

Decizia 5

if (len >= 3)

Condiția individuală:

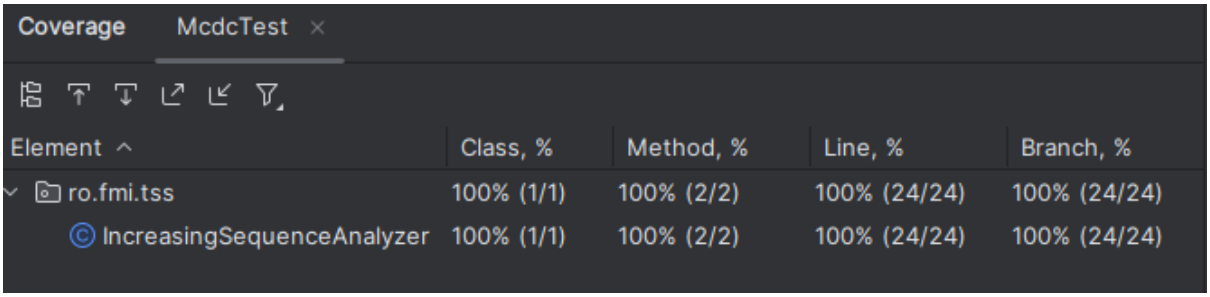
C: len >= 3

	C	D5
T12	true	true
T13	false	false

T3 acopera T12
T13: [10, 20, 10, 20, 10, 20] (lugimea tuturor secventelor este de 2)

Setul final de teste care satisface criteriul *modified condition/decision coverage (MC/DC)*:

- analyze(new int[]{10, 20, 30, 40, 50}) -> returnează "Vectorul nu este valid" (Acoperă T1)
- analyze(new int[]{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13}) -> returnează "Vectorul nu este valid" (Acoperă T2)
- analyze(new int[]{10, 20, -5, 40, 50, 60}) -> returnează "Vectorul nu este valid" (Acoperă T4)
- analyze(new int[]{10, 20, 105, 40, 50, 60}) -> returnează "Vectorul nu este valid" (Acoperă T5)
- analyze(new int[]{10, 20, 50, 5, 15, 30}) -> returnează "DA" (Acoperă T3)
- analyze(new int[]{10, 20, 30, 40, 50, 90}) -> returnează "NU" (Acoperă T8)
- analyze(new int[]{10, 20, 30, 5, 10, 20}) -> returnează "NU" (Acoperă T9)
- analyze(null) -> returnează "Vectorul nu este valid" (Acoperă T10)
- analyze(new int[]{10, 20, 10, 20, 10, 20}) -> returnează "NU" (Acoperă T13)



Exercițiul 5

Mutant de ordinul 1 echivalent al programului

Modificăm operatorul relațional `<` (mai mic) în `<=` (mai mic sau egal), ajustând și valoarea cu 1. Așadar instrucțiunea de la linia 5 **“if (scores.length < 6 || scores.length > 12) { “** va deveni **“if (scores.length <= 5 || scores.length > 12) { ”**.

Cerinta 6 - Mutanti neechivalenti

1. Cazul de test ales

Pentru aceasta cerinta a fost ales testul **T1 (`testC1`)** din setul Equivalence Partitioning, realizat la cerinta 1.

Intrare: `[10, 30, 50, 0, 20, 70]`

Rezultat asteptat: `"DA"`

Vectorul contine doua secvente strict crescatoare relevante, cu ultimele valori 50 si 70.

Astfel, `seqCount = 2`, iar `sum = 120`.

Testul trece pe implementarea originala.

2. Mutantul M1 - Killed

In implementarea originala, conditia finala este:

```
if (seqCount >= 2 && sum >= 80)
```

Pentru M1, operatorul `>=` a fost inlocuit cu `>`:

```
if (seqCount > 2 && sum >= 80)
```

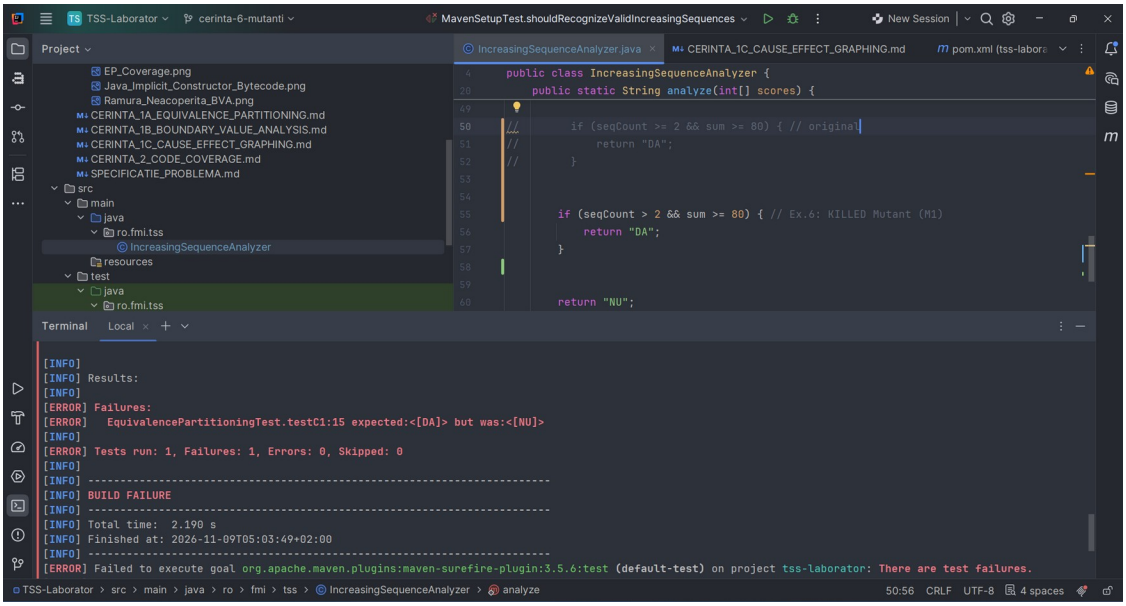
Mutatia modifica o singura conditie relationala.

Pentru intrarea T1:

- Programul original returneaza `"DA"`.
- Mutantul M1 returneaza `"NU"`.

La executarea testului T1, JUnit raporteaza **1 test esuat** deoarece rezultatul obtinut difera de cel asteptat.

Prin urmare, mutantul M1 este **neechivalent si omorat (Killed)** de T1.



Mutantul M1 omorat de T1

3. Mutantul M2 - Survived

Pornind din nou de la implementarea originala, a fost modificata conditia de validare a scorurilor:

Original: `scores[i] > 100`

Mutant M2: `scores[i] > 101`

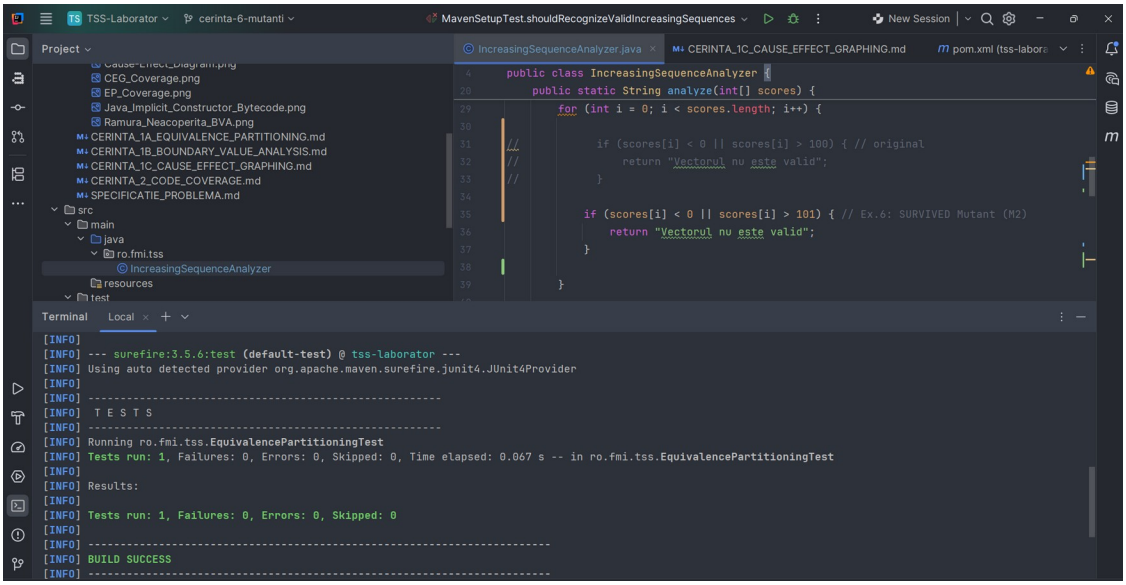
Aceasta mutatie permite in mod eronat valoarea 101, care este interzisa conform specificatiei problemei.

La executarea aceluiasi test T1, toate scorurile sunt in intervalul valid si modificarea nu influenteaza rezultatul.

- Programul original returneaza `"DA"`.
- Mutantul M2 returneaza `"DA"`.

JUnit raporteaza **1 test trecut, fara erori**.

Prin urmare, mutantul M2 **supravietuieste (Survived)** testului T1.



Mutantul M2 supravietuieste testului T1

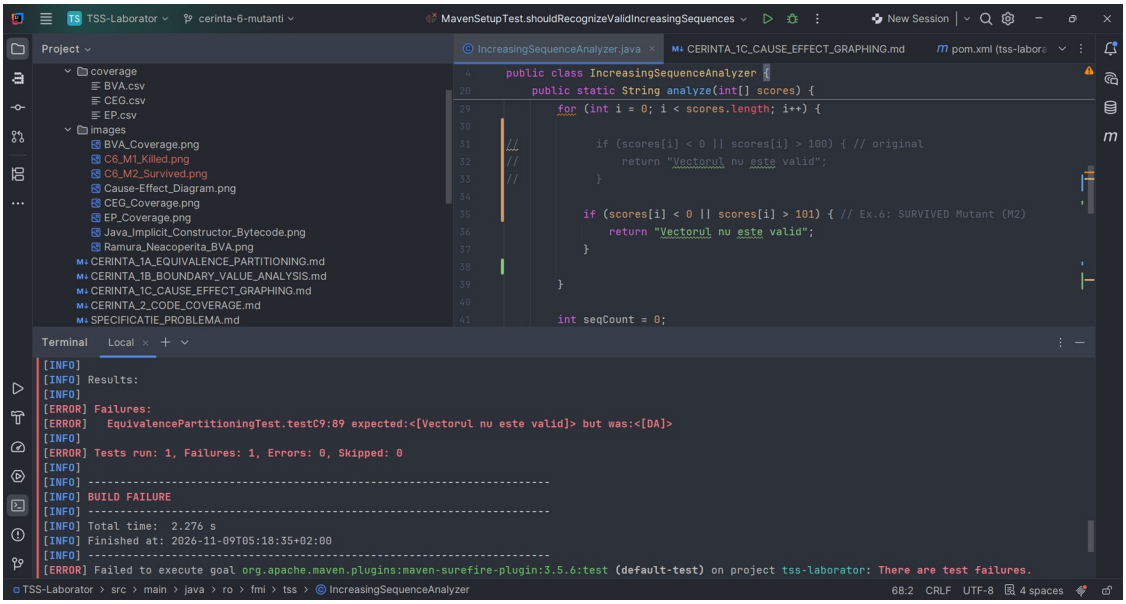
Demonstrarea neechivalentei mutantului M2

Pentru a demonstra ca M2 nu este echivalent cu programul original, a fost folosit suplimentar testul T9 (`testC9`) din Equivalence Partitioning.

Intrare: `[10, 30, 50, 0, 20, 101]`

- **Original:** `"Vectorul nu este valid"` deoarece scorul 101 depaseste limita 100.
- **M2:** `"DA"` deoarece noua conditie accepta valoarea 101.

Testul T9 esueaza pe M2, demonstrand ca mutantul produce un comportament diferit fata de programul original.



Demonstrarea neechivalentei M2

T9 a fost utilizat numai pentru demonstrarea neechivalentei. Testul de referinta pentru cei doi mutanti ramane T1.

4. Compararea rezultatelor

Program	Rezultat pentru T1	Verdict
Original	DA	Test trecut
M1 (`>= 2` devine `> 2`)	NU	Killed
M2 (`> 100` devine `> 101`)	DA	Survived

Ambii mutanti sunt neechivalenti cu programul original deoarece exista intrari pentru care comportamentul lor difera de acesta.

5. Concluzie

A fost utilizat acelasi caz de test T1 pentru evaluarea a doi mutanti neechivalenti, construiti manual prin modificarea unui singur operator relational.

Testul T1 a omorat mutantul M1, dar nu a detectat mutantul M2. Neechivalenta lui M2 a fost demonstrata separat prin testul T9.

Rezultatele evidentiaza faptul ca un test poate detecta anumite defecte introduse in program, fara a le detecta pe toate.

Exercitiul 7

Tool PITest

Exercițiul 1

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
1	96%	25/26	97%	36/37	97%	36/37

Breakdown by Package

Name	Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
ro.fmi.tss	1	96%	25/26	97%	36/37	97%	36/37

Report generated by PIT 1.15.0

Enhanced functionality available at arcmutate.com

MC/DC (Exercițiul 4)

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
1	96%	25/26	86%	32/37	86%	32/37

Breakdown by Package

Name	Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
ro.fmi.tss	1	96%	25/26	86%	32/37	86%	32/37

Report generated by PIT 1.15.0

Enhanced functionality available at arcmutate.com

	Tehnici Utilizate	Mutanti Generati	Mutanti Omorati	Mutanti Supravietuitori	Mutation Score
Teste Exercițiul 1	Black-box(BVA, EP, CEG)	37	36	1	97%

Teste Exercițiul 4	White-box(MC/DC)	37	32	5	86%

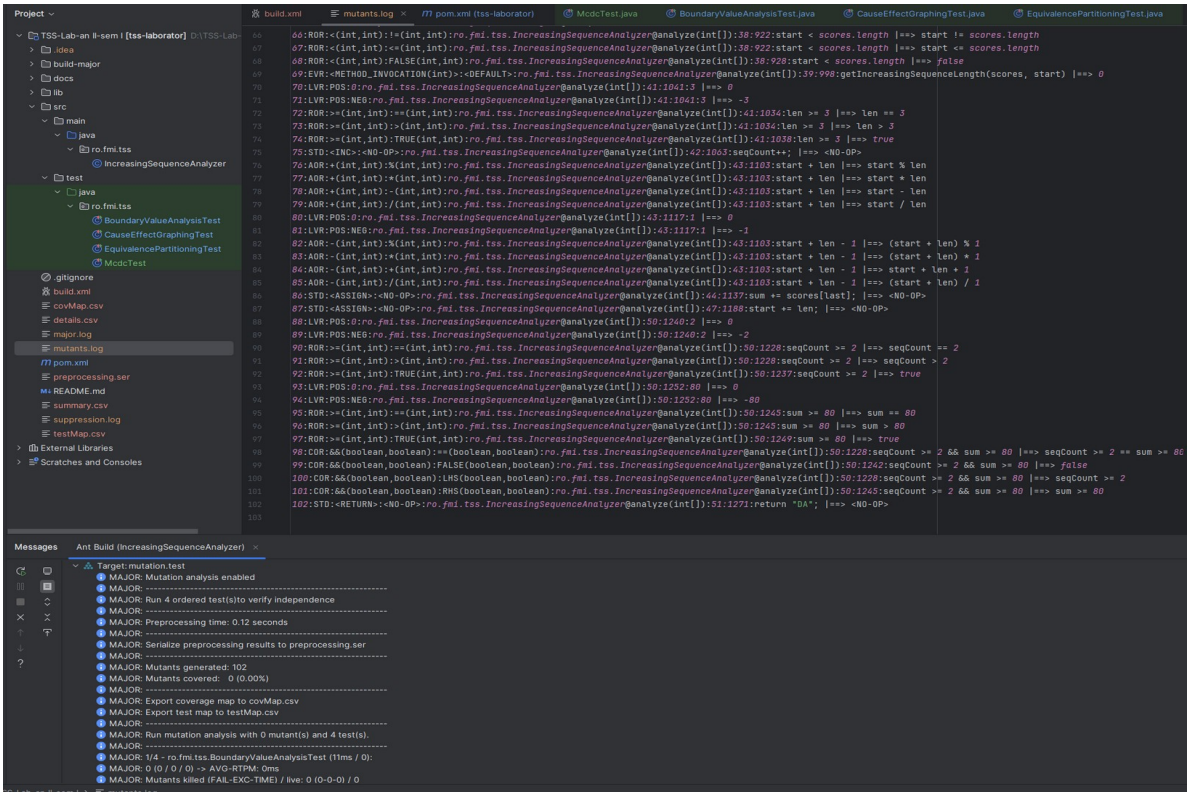
Analiza și Justificarea Diferențelor

- **Eficiența suitei Black-Box (Exercițiul 1):** Scorul de 36/37 se datorează includerii **Analizei Valorilor de Frontieră (BVA)**. PITest generează automat un număr mare de mutanți de graniță (ex. *Conditional Boundary Mutator*, care transformă operatorul < în <=). Deoarece tehnica BVA obligă proiectarea cazurilor de testare exact pe limitele intervalelor (ex. vectori de lungime 5 și 6; scoruri de -1, 0, 100, 101), acești mutanți relaționali au fost detectați și „omorâți” imediat. Singurul mutant supraviețuitor este, foarte probabil, un mutant echivalent.
- **Limitările suitei MC/DC (Exercițiul 4):** Criteriul MC/DC este proiectat pentru a demonstra influența independentă a fiecărei sub-condiții booleene asupra deciziei finale, folosind un număr minimizat de teste (9 teste). Deși MC/DC garantează acoperirea structurii logice, **nu obligă testarea pe valorile de graniță**. Pentru a acoperi condiția “scores.length < 6”, un test cu len = 3 (Adevărat) și unul cu len = 8 (Fals) satisfac criteriul MC/DC. Totuși, ambele teste vor trece cu succes și pe codul mutat “scores.length <= 6”. Astfel, cei 5 mutanți care au supraviețuit testelor MC/DC sunt mutanți de graniță pe care suita i-a ratat din cauza datelor de intrare prea îndepărtate de extremitățile intervalelor matematice.

Concluzia Analizei

Rezultatele demonstrează că **nicio tehnică de testare nu este perfectă în mod izolat**. Criteriul MC/DC este esențial pentru validarea riguroasă a structurii logice (AND/OR) cu un minim de efort, însă trebuie întotdeauna complementat de Analiza Valorilor de Frontieră (BVA) pentru a securiza limitele algebrice și a preveni defectele relaționale.

Tool Major



Concluzia Analizei

Nivelul de aplicare a mutațiilor: PITest acţionează la nivel de *bytecode* (cod compilat), aplicând un set restrâns şi optimizat de mutaţii pentru a reduce timpul de execuţie. Major acționează direct pe *codul sursă* (Abstract Syntax Tree - AST), având acces la un set mult mai extins de operatori clasici de mutație (AOR - înlocuire operatori aritmetici, ROR - înlocuire operatori relaționali, COR - înlocuire operatori condiționali).

Volumul de mutanți și mutanții echivalenți: Datorită setului extins de operatori pe AST, Major genereaza 102 mutanti, număr considerabil mai mare de mutanți pentru metoda *analyze* comparativ cu cei 37 generați de PITest. Acest volum vine cu un dezavantaj major: introducerea unei rate mult mai mari de **mutanți echivalenți** (mutanţi care modifică sintaxa, dar nu schimbă absolut deloc logica sau rezultatul programului, fiind imposibil de „omorât”).

Contributii individuale

Cerinta	Economu Victor	Radu Alexandru-Mihail
1. EP, BVA, CEG	100%	0%
2. Code Coverage (JaCoCo)	100%	0%
3. Graf CFG si complexitate ciclomatica	0%	100%
4. MC/DC	0%	100%
5. Mutant echivalent de ordinul 1	0%	100%
6. Mutanti neechivalenti	100%	0%
7. Mutation testing: PIT si Major (partial)	0%	100%

La cerinta 7, analiza PIT a fost documentata, iar analiza Major a ramas partial documentata.

Ponderea contributiei la proiectul total

Student	Procent din proiect
Economu Victor	50%
Radu Alexandru-Mihail	50%
Total	100%

Utilizarea instrumentelor AI (AI Usage Log)

Au fost folosite instrumente AI ca suport pentru clarificari tehnice si redactare. Solicitarile sunt descrise pe scurt.

1. Implementarea programului Java si a testelor

Instrument AI: ChatGPT

Scop: Sprijin pentru codul Java al algoritmului IncreasingSequenceAnalyzer, validarea intrarilor si proiectarea testelor JUnit.

Prompt / descrierea solicitarii: Solicitari privind implementarea algoritmului, aplicarea tehnicilor de testare conform cerintelor laboratorului.

Rezultat obtinut: Sugestii de implementare, exemple de teste Junit.

Modificari efectuate de echipa: Victor a selectat si adaptat datele de test la specificatie, a introdus testele in proiect, a realizat experimentele cu mutanti si a restaurat codul original.

Verificare si validare: Executarea testelor JUnit (39 de teste initiale, fara erori), compararea cu specificatia si rularea separata a mutantilor, cu dovezi de tip Killed / Survived.

2. Reformulari academice si structurarea documentatiei

Instrument AI: ChatGPT si Gemini

Scop: Imbunatatirea claritatii formularilor tehnice.

Prompt / descrierea solicitarii: Solicitari de reformulare in stil academic si de organizare a descrierilor pentru metodele de testare, JaCoCo si mutatii, fara schimbarea sensului tehnic.

Rezultat obtinut: Propuneri de paragrafe.

Modificari efectuate de echipa: Studentii au selectat si revizuit formularea, au verificat vectorii si rezultatele, au construit diagramele si au corectat manual tabelele si asezarea in pagina.

Verificare si validare: Compararea cu enuntul oficial si cu rapoartele generate (JUnit / JaCoCo), verificarea imaginilor, formulelor si tabelelor in documentul final.

3. Asistenta la diagnosticarea integrarii Major

Instrument AI: Gemini

Scop: Identificarea problemelor de compilare, executie si colectare a acoperirii mutantilor pentru Major, Apache Ant si JUnit.

Prompt / descrierea solicitarii: Solicitari privind erorile Java 11, configurarea major.jar / major-rt.jar, selectarea testelor si investigarea rezultatului „Mutants covered: 0”.

Rezultat obtinut: Propuneri de configurare Ant, ajustari de classpath si experimente de diagnosticare pentru executia cu JUnit 4.

Modificari efectuate de echipa: Radu a adaptat local configuratia build.xml si bibliotecile si a incercat verificarile sugerate.

Verificare si validare: Au fost raportati 102 mutanti generati si executarea claselor de test. Colectarea acoperirii mutantilor a ramas neconfirmata; nu este prezentat un mutation score Major ca rezultat valid.